Training Dynamics of Overparameterized Neural Networks

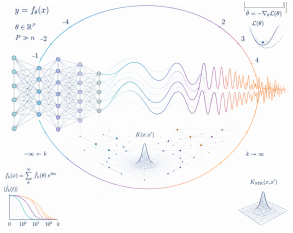
Shreyas Kalvankar

MSc Computer Science, TU Delft

Advisor: Prof. Dr. D. M. J. Tax, Pattern Recognition & Bioinformatics Group
Supervisor: Prof. Dr. A. Heinlein, Numerical Analysis Group

Committee Member: Prof. Dr. A. Papapantoleon, Applied Probability Group

TU Delft



Spectral Bias | NTK on S^1

Fourier Analysis

Finite-Scale Bounds

INTRODUCTION

Supervised Learning: Fitting data to a function

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

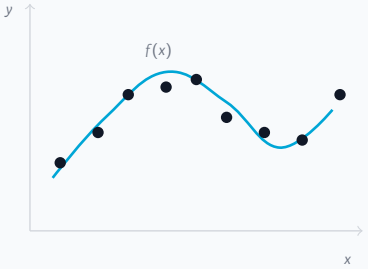
Model: a learned prediction function $f(x)$.

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.



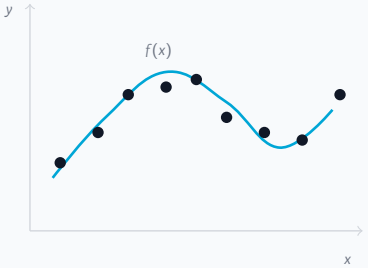
Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n}\sum_{i=1}^{n}\big(\underbrace{f(x_i) - f^\star(x_i)}_{:= r_i}\big)^2$.



Supervised Learning: Fitting data to a function

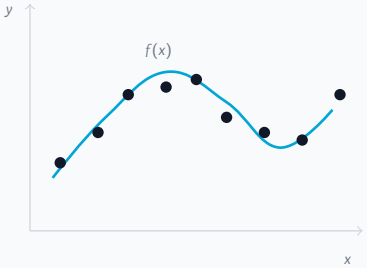
Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n}\sum_{i=1}^{n}\big(\underbrace{f(x_i) - f^\star(x_i)}_{:= r_i}\big)^2$.

In practice: we cannot search over all possible functions, so we fix a parameterized family $f_\theta(x)$ and learn by adjusting the parameters θ. The parameters are the unknowns the data must pin down.



MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ:

$$f_\theta(x_i) \approx y_i, \text{ for } i = 1, \ldots, n.$$

A *solution*: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ:

$$f_\theta(x_i) \approx y_i, \text{ for } i = 1, \ldots, n.$$

A *solution*: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

Large networks can also fit random labels, so fitting the training set alone does not explain generalization (Zhang et al., 2017).

Empirical observation: models often learn simple structure before complex detail (Rahaman et al., 2019; Boursier & Flammarion, 2024).

MOTIVATION & RELEVANCE

Why study training behaviour?

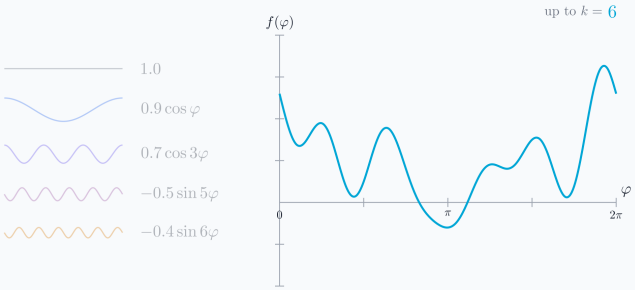
The broader goal is to understand learning, not just report a final test error.

- Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).
- So before asking for a bigger model, we can ask: **what does gradient descent learn first?**

ILLUSTRATION

A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



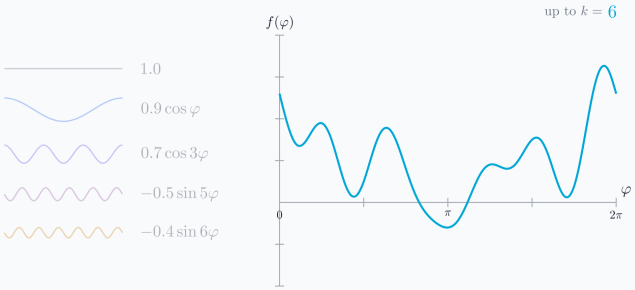
Frequency components

- Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- Low frequencies set the coarse shape; high frequencies add the fine detail.
- The target is a short sum of these pieces.

ILLUSTRATION

A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



Frequency components

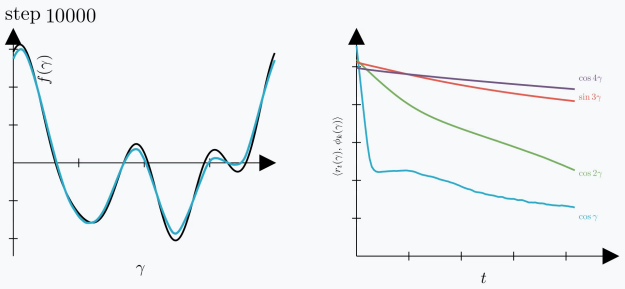
- Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- Low frequencies set the coarse shape; high frequencies add the fine detail.
- The target is a short sum of these pieces.

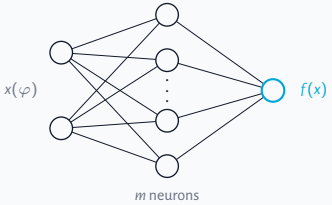
What gets learned first?

ILLUSTRATION

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?





$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right), \quad \sigma = \mathrm{ReLU}$$

The coarse shape appears first; the fine detail comes last.

Notice

- The fit climbs toward the target.
- Low-frequency modes die first.
- Fine detail fills in last.

THESIS

Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

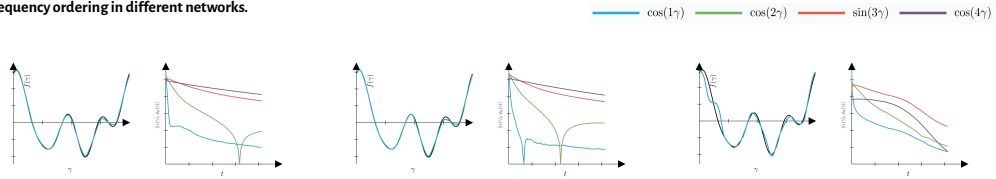
Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Three questions

1. What sets the learning speed of each frequency?
2. Does the answer survive when the network sees only n sampled points? *(RQ1)*
3. Does it survive with finitely many neurons, at the start of training and during it? *(RQ2)*

Frequency ordering in different networks.

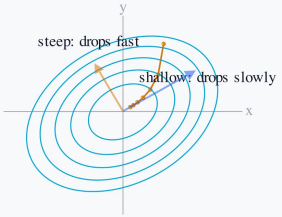


BACKGROUND

The error is a landscape

Seen from above, the contour rings are the bowl's height; training rolls toward the bottom.

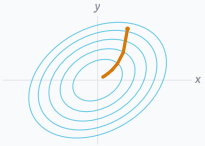
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i \left(f(x_i) - f^*(x_i)\right)^2$



Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$

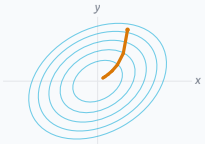


In x, y axes the two directions are coupled.

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

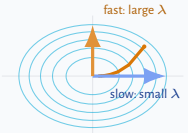
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



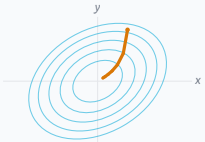
Along each axis the error falls on its own.

BACKGROUND

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

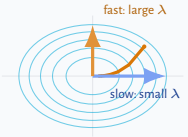
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

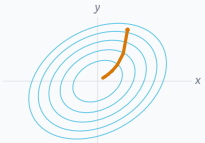
Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

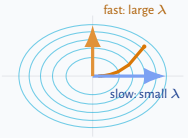
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

so it decays exponentially:

$$\alpha(t) = \alpha(0)\,e^{-\lambda t}$$

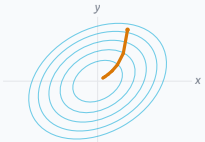
like cooling or half-life; λ is the rate.

BACKGROUND

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

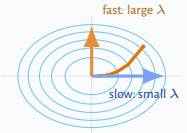
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

so it decays exponentially:

$$\alpha(t) = \alpha(0)\,e^{-\lambda t}$$

like cooling or half-life; λ is the rate.

Eigenvector: a direction the dynamics keeps separate, never mixing it with others. **Eigenvalue** λ: its rate, large is fast, small is slow.

What are these directions and rates for a neural network?

BACKGROUND

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^\star(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

The error falls at rate λ, a single number.

A whole function

$$\partial_t r_t = -T_t\, r_t$$

The rate becomes an operator T_t, the network's tangent kernel (NTK).

If T_t were time-independent, diagonalizing it would split this back into one copy of the scalar law per direction k:

$$\dot{\alpha}_k = -\lambda_k\,\alpha_k$$

eigenfunctions are the directions · **eigenvalues** λ_k are the rates · together they form the **spectrum** of the time-independent T

So the question becomes: what is the spectrum of T?

SETUP

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

recall: $\partial_t r_t = -T_t\, r_t$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

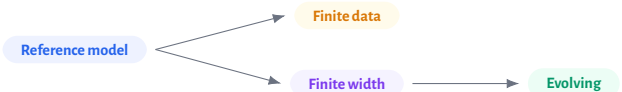
Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Solution: Consider using the continuum as input. (Jacot et al., 2018, Cao et al., 2019).

Strategy: build the clean reference, then relax each idealization separately.



SETUP

Setting: regression on S^1

S^1: functions on the circle have a natural Fourier decomposition.

SETUP

Setting: regression on S^1

S^1: functions on the circle have a natural Fourier decomposition.

Domain + target

$$x(\varphi) = (\cos\varphi, \sin\varphi) \in S^1$$

$$y(\varphi) = \sum_k c_k \phi_k(\varphi)$$

Fourier basis: $1, \cos(k\varphi), \sin(k\varphi)$.

SETUP

Setting: regression on S^1

S^1: functions on the circle have a natural Fourier decomposition.

Domain + target

$$x(\varphi) = (\cos\varphi, \sin\varphi) \in S^1$$

$$y(\varphi) = \sum_k c_k \phi_k(\varphi)$$

Fourier basis: $1, \cos(k\varphi), \sin(k\varphi)$.

Network

$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right)$$

- one hidden layer of width m, ReLU σ
- Gaussian init; biases start at 0, all of θ trains
- $1/\sqrt{m}$: NTK scaling

SETUP

Setting: regression on S^1

S^1: functions on the circle have a natural Fourier decomposition.

Domain + target

$$x(\varphi) = (\cos\varphi, \sin\varphi) \in S^1$$

$$y(\varphi) = \sum_k c_k \phi_k(\varphi)$$

Fourier basis: $1, \cos(k\varphi), \sin(k\varphi)$.

Network

$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right)$$

- one hidden layer of width m, ReLU σ
- Gaussian init; biases start at 0, all of θ trains
- $1/\sqrt{m}$: NTK scaling

Training dynamics

$$\partial_t r_t = -T_t^{(m)} r_t$$

$$r_t = f_t - y$$

$T_t^{(m)}$ is the tangent operator.

SETUP

Setting: regression on S^1

S^1: functions on the circle have a natural Fourier decomposition.

Domain + target

$$x(\varphi) = (\cos\varphi, \sin\varphi) \in S^1$$

$$y(\varphi) = \sum_k c_k \phi_k(\varphi)$$

Fourier basis: $1, \cos(k\varphi), \sin(k\varphi)$.

Network

$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right)$$

- one hidden layer of width m, ReLU σ
- Gaussian init; biases start at 0, all of θ trains
- $1/\sqrt{m}$: NTK scaling

Training dynamics

$$\partial_t r_t = -T_t^{(m)} r_t$$

$$r_t = f_t - y$$

$T_t^{(m)}$ is the tangent operator.

Key question: what governs the decay of each mode $\alpha_k(t) = \langle r_t, \phi_k \rangle$?

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

$$T_\infty \phi_k = \lambda_k \phi_k$$

Fourier modes are eigenfunctions

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

$$T_\infty \phi_k = \lambda_k \phi_k$$

Fourier modes are eigenfunctions

$$\alpha_k(t) = e^{-\lambda_k t}\alpha_k(0)$$

rate = eigenvalue

REFERENCE MODEL

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

$$T_\infty \phi_k = \lambda_k \phi_k$$

Fourier modes are eigenfunctions

$$\alpha_k(t) = e^{-\lambda_k t}\alpha_k(0)$$

rate = eigenvalue

Explicit eigenvalues Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

$$T_\infty \phi_k = \lambda_k \phi_k$$

Fourier modes are eigenfunctions

$$\alpha_k(t) = e^{-\lambda_k t}\alpha_k(0)$$

rate = eigenvalue

Explicit eigenvalues Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- Small k have larger eigenvalues, so low frequencies decay faster.
- For large k, $\lambda_k \sim k^{-2}$, so high frequencies are delayed.

The ideal picture: infinite-width MLP on S^1

For uniform data on the circle, the NTK is a fixed convolution operator, so the Fourier modes are its eigenfunctions.

$$\partial_t r_t = -T_\infty r_t$$

fixed NTK integral operator

$$T_\infty \phi_k = \lambda_k \phi_k$$

Fourier modes are eigenfunctions

$$\alpha_k(t) = e^{-\lambda_k t}\alpha_k(0)$$

rate = eigenvalue

Explicit eigenvalues Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- Small k have larger eigenvalues, so low frequencies decay faster.
- For large k, $\lambda_k \sim k^{-2}$, so high frequencies are delayed.

Reference model: exact Fourier dynamics at infinite width and in the continuum.

Jacot et al. (2018)

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand its perturbations.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand its perturbations.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand its perturbations.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Frozen finite width

$$T_\infty \quad \rightsquigarrow \quad T_0^{(m)}$$

Replace the infinite-width operator by the finite-width tangent operator at initialization.

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand its perturbations.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Frozen finite width

$$T_\infty \quad \rightsquigarrow \quad T_0^{(m)}$$

Replace the infinite-width operator by the finite-width tangent operator at initialization.

Question

Does the finite-width tangent operator preserve the Fourier eigenspaces and learning rates?

Both perturbations break exact diagonalization; the goal is to quantify the finite-scale error. cf. Bowman & Montúfar (2022)

Main results: the spectral picture survives finite scale

Fix a low-frequency window: the Fourier modes up to frequency K, $d = 2K + 1$ of them. Compare each finite-scale object to its ideal counterpart there.

Finite samples

Thm. 5.5

Train on n sampled points instead of the whole circle. On the window, the sampled Fourier geometry (orthogonality, eigenvalues, kernel action) deviates from the ideal one by at most

$$C\sqrt{\frac{\log(nd/\delta)}{n}}$$

with probability $1 - \delta$, for explicit constants C.

Main results: the spectral picture survives finite scale

Fix a low-frequency window: the Fourier modes up to frequency K, $d = 2K + 1$ of them. Compare each finite-scale object to its ideal counterpart there.

Finite samples

Thm. 5.5

Train on n sampled points instead of the whole circle. On the window, the sampled Fourier geometry (orthogonality, eigenvalues, kernel action) deviates from the ideal one by at most

$$C\sqrt{\frac{\log(nd/\delta)}{n}}$$

with probability $1 - \delta$, for explicit constants C.

Frozen finite width

Thm. 5.6

Replace the ideal operator by a width-m network's tangent operator at initialization. On the window it equals the ideal one *on average*, and deviates by at most

$$C\sqrt{\frac{\log(d/\delta)}{m}}$$

with probability $1 - \delta$; on average: $\mathbb{E}[\text{finite-width block}] = \text{ideal block}$.

Main results: the spectral picture survives finite scale

Fix a low-frequency window: the Fourier modes up to frequency K, $d = 2K + 1$ of them. Compare each finite-scale object to its ideal counterpart there.

Finite samples

Thm. 5.5

Train on n sampled points instead of the whole circle. On the window, the sampled Fourier geometry (orthogonality, eigenvalues, kernel action) deviates from the ideal one by at most

$$C\sqrt{\frac{\log(nd/\delta)}{n}}$$

with probability $1 - \delta$, for explicit constants C.

Frozen finite width

Thm. 5.6

Replace the ideal operator by a width-m network's tangent operator at initialization. On the window it equals the ideal one *on average*, and deviates by at most

$$C\sqrt{\frac{\log(d/\delta)}{m}}$$

with probability $1 - \delta$; on average: $\mathbb{E}[\text{finite-width block}] = \text{ideal block}$.

Takeaway: finite-sample errors shrink like $O(n^{-1/2})$, frozen finite-width errors like $O(m^{-1/2})$, both on the same fixed low-frequency subspace: the spectral picture degrades gracefully.

Rates up to log factors; precise statements and constants in backup. Finite scale vs. idealized kernel dynamics: Bowman & Montúfar (2022).

EVOLVING KERNEL

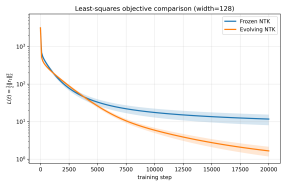
A finite-width effect not explained by the frozen NTK

After controlling the frozen operator, the next question is whether training changes the operator enough to matter.

EVOLVING KERNEL

A finite-width effect not explained by the frozen NTK

After controlling the frozen operator, the next question is whether training changes the operator enough to matter.

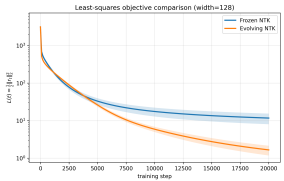


$m = 128$: evolving reaches much lower loss

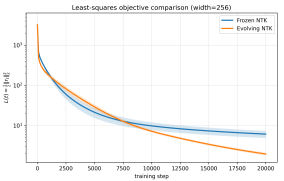
EVOLVING KERNEL

A finite-width effect not explained by the frozen NTK

After controlling the frozen operator, the next question is whether training changes the operator enough to matter.



$m = 128$: evolving reaches much lower loss

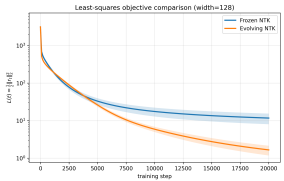


$m = 256$: evolving still wins

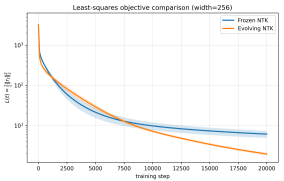
EVOLVING KERNEL

A finite-width effect not explained by the frozen NTK

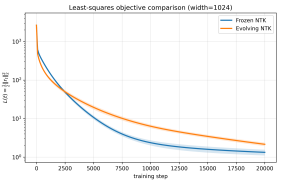
After controlling the frozen operator, the next question is whether training changes the operator enough to matter.



$m = 128$: evolving reaches much lower loss



$m = 256$: evolving still wins



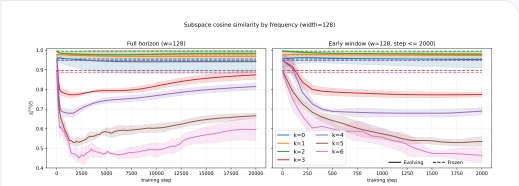
$m = 1024$: frozen reaches lower final loss

- Small width: moving the kernel helps, so frozen NTK misses something.
- Large width: frozen wins again, matching the lazy-training picture.
- The reversal forces the question: what changed in the moving kernel?

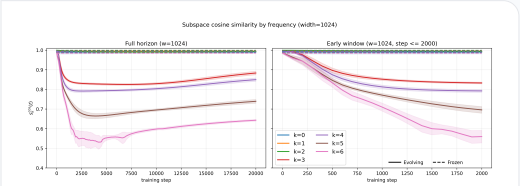
EVOLVING KERNEL

First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise. They do not.



$m = 128$: higher-frequency alignment drops sharply.

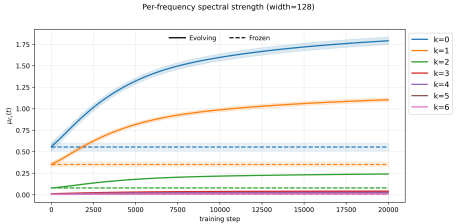


$m = 1024$: frozen alignment starts stronger, but evolution still worsens high frequencies.

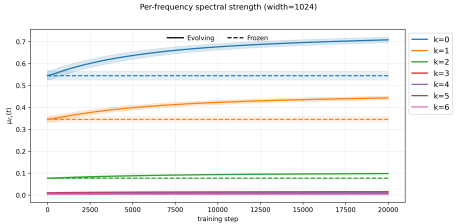
Alignment is not the gain. The moving kernel does not become more Fourier-diagonal.

What changes: low-frequency strength rises

The benefit tracks operator strength on the low-frequency blocks, not better alignment.



$m = 128$: large boost in $k = 0$ and $k = 1$.



$m = 1024$: same pattern, much weaker.

Spectral bias is reinforced. What breaks is the frozen description, not the low-to-high ordering.

Correlation with lower loss, not established cause. Small-width effect: at large width the frozen picture wins ($m = 1024$), as lazy training predicts.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 Rates

Mode-wise rates are eigenvalues. Low frequencies have larger eigenvalues and are learned first; $\lambda_k \sim k^{-2}$.

2 Finite data

Finite samples and frozen finite width preserve the picture, with explicit $O(n^{-1/2})$ and $O(m^{-1/2})$ control.

3 Finite width

Training evolves the kernel and reinforces the low-frequency bias. The frozen description holds where the kernel stays lazy, at large width.

SCOPE

What is special about the circle, and what is not

Special

Uniform measure on S^1 makes the NTK a convolution, so its eigenfunctions are exactly the Fourier modes.

Carries over

Off this setting (non-uniform density, or Gaussian inputs giving Hermite polynomials) the basis changes, but eigenvalue size still sets the rates.

Open problem

A non-asymptotic theory for the evolving kernel: control of $T_t^{(m)} - T_0^{(m)}$ over training time.

Fourier structure survives finite scale, but kernel evolution changes the story.

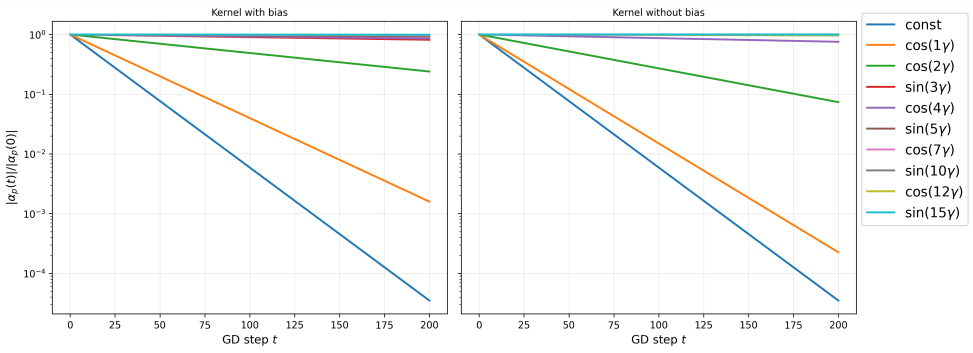
In the lazy regimes it predicts the rates; at small width, training actively reinforces the low-frequency bias by increasing low-frequency operator strength.

Questions?

[acknowledgements placeholder]

Backup

Continuum prediction: spectral bias



NTK: from parameter updates to an operator on functions

[Backup] Derivation of the NTK: how gradient descent on weights becomes $\partial_t r_t = -T_t r_t$

Finite-scale bounds: precise statements

$\mathcal{H}_K$: span of the Fourier modes up to frequency K, $d = 2K + 1$; Φ: those modes evaluated at the n samples; $\|\cdot\|_{\max}$: entrywise max.

Finite samples

Thm. 5.5

Sampled kernel, Gram, and compressed operator:

$$A_{ij} = \tfrac{1}{n} T_\infty(\theta_i, \theta_j), \qquad G = \tfrac{1}{n}\Phi^\top \Phi, \qquad H = \tfrac{1}{n}\Phi^\top A \Phi$$

With probability at least $1 - \delta$:

$$\|G - I\|_{\max} \le C_1 \sqrt{\frac{\log(d/\delta)}{n}} \qquad \text{(orthogonality)}$$

$$\|H - \Lambda^{(n)}\|_{\max} \le C_2 \sqrt{\frac{\log(d/\delta)}{n}} \qquad \text{(eigenvalues)}$$

$$\|A\Phi - \Phi\Lambda^{(n)}\|_{\max} \le C_3 \sqrt{\frac{\log(nd/\delta)}{n}} \qquad \text{(kernel action)}$$

$\Lambda^{(n)}$: ideal eigenvalues with an $O(1/n)$ correction; $\mathbb{E}[H] = \Lambda^{(n)}$ (Prop. 5.4).

Frozen finite width

Thm. 5.6

Compress the width-m tangent operator at initialization to $\mathcal{H}_K$:

$$B_m^{(K)} = P_K T_0^{(m)} P_K, \qquad \Lambda_K = P_K T_\infty P_K$$

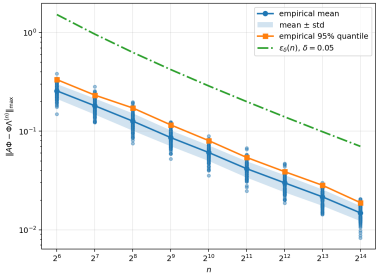
Unbiased at every width, and concentrating:

$$\mathbb{E}\, B_m^{(K)} = \Lambda_K$$

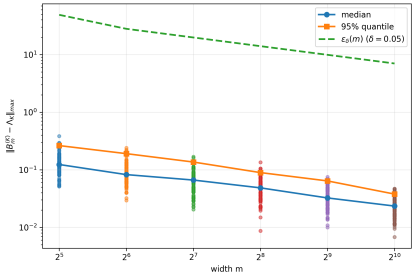
$$\|B_m^{(K)} - \Lambda_K\|_{\max} \le C_4 \sqrt{\frac{\log(d/\delta)}{m}}$$

with probability at least $1 - \delta$; sub-exponential regime omitted.

Concentration bounds: the constants, and where the magnitude bound is loose

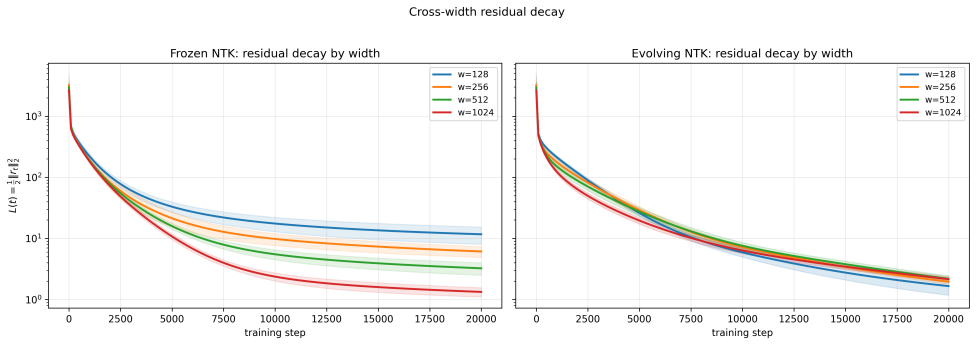


finite samples: action error decreases with n

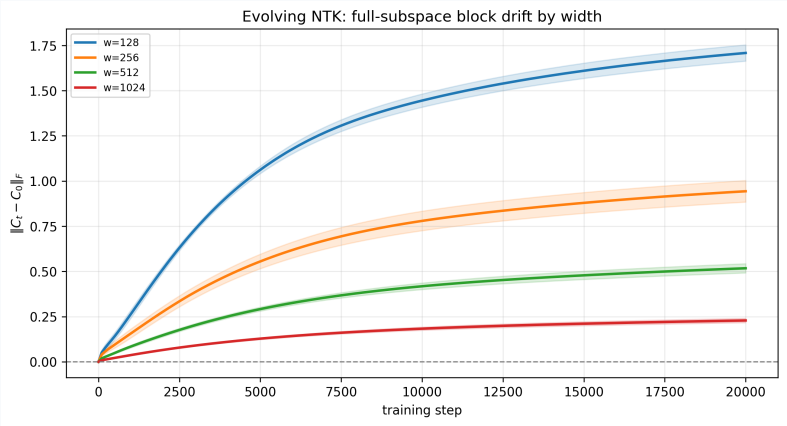


frozen finite width: block error decreases with m

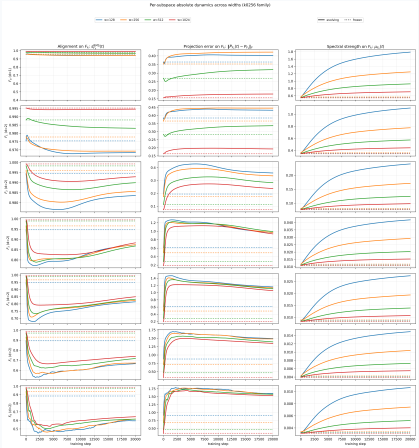
Frozen versus evolving: cross-width view



Kernel movement is a small-width effect



Full per-subspace evolving-kernel diagnostics



Computing the eigenvalues λ_k

[Backup] Integral computation of λ_k for the ReLU NTK on S^1; verification against the explicit cases $k = 0, 1$

The $k = 2$ anomaly

[Backup] Why λ_2 does not follow the generic even-k formula; how the $k^2 - 1$ denominator is modified at $k = 2$

Strict ordering: where it holds and where it does not

[Backup] Conditions under which $\lambda_k > \lambda_{k+1}$; counterexamples or caveats near the transition between even and odd cases

Why $k = 0$ and $k = 1$ gain strength: a hypothesis and its test

[Backup] Parameterisation hypothesis; proposed clean test (target with no $k = 0, 1$ component)

Mean-field and preconditioning views

[Backup] Mean-field interpretation of infinite-width limit; preconditioning perspective on spectral bias

References

- Rahaman, Baratin, Arpit, Draxler, Lin, Hamprecht, Bengio, Courville (2019). *On the Spectral Bias of Neural Networks*. ICML.
- Jacot, Gabriel, Hongler (2018). *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS.
- Bowman, Montúfar (2022). *Spectral Bias Outside the Training Set for Deep Networks in the Kernel Regime*. NeurIPS.
- Zhang, Bengio, Hardt, Recht, Vinyals (2017). *Understanding Deep Learning Requires Rethinking Generalization*. ICLR.